

Marat Bekzat
(Tuesday 17:00-18:00)

Practice 1

Part 1

Task 1.1

Relation A

1. Superkeys : EmpID , SSN , Email , Phone , EmpID / Name , EmpID / Department , SSN / Name , Email / Phone.
2. Candidate keys: EmpID , SSN , Email , Phone.
3. Primary key : EmpID. Reason: the EmpID is a short and easy to find. Also it does not using a personal information.
4. No they can not have the same number. But it can not be a unique because people can change their number. It can be a Candidate key but not a primary.

Relation B

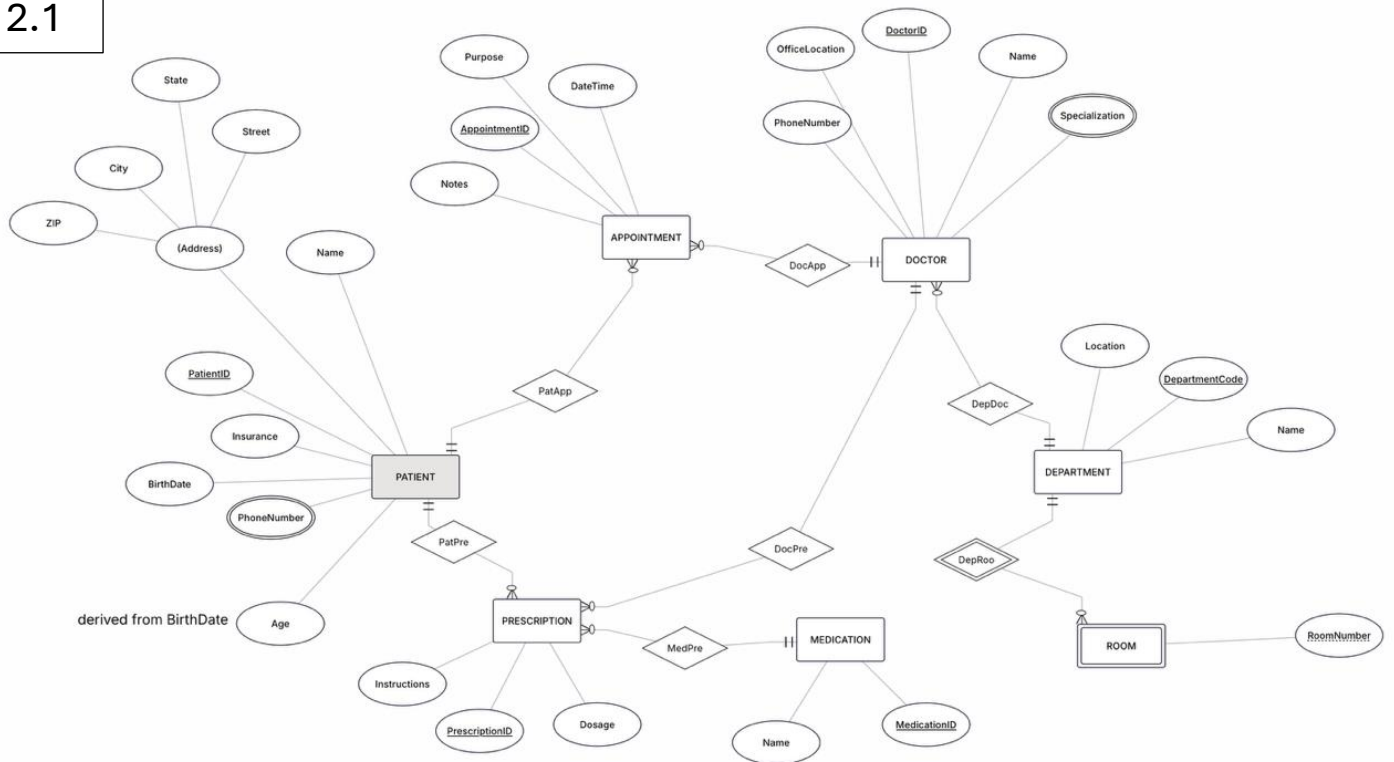
1. Primary key : Student ID, Course Code, Section, Semester, Year.
2. 1) Student ID – The best choice for the primary key, because it's unique, easy to remember and identifies the student.
2) CourseCode – Identifies the course.
3) Selection – Identifies the course group.
3. Candidate keys : Semester / Year.

Task 1.2

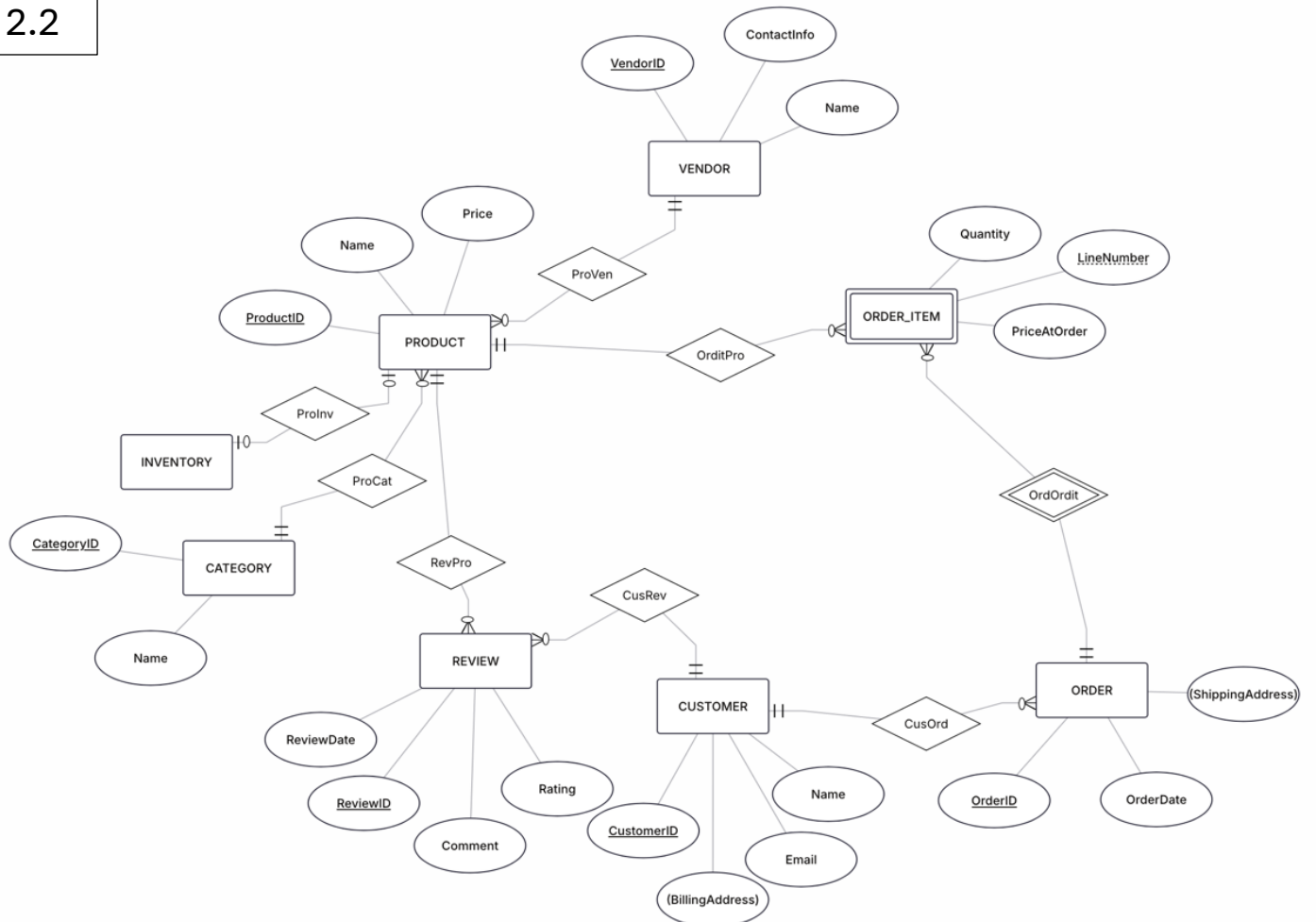
1. Student(AdvisorID) / Professor(ProfID)
2. Professor(Department) / Department(DeptCode)
3. Course(DepartmentCode) / Department(DeptCode)
4. Enrollment(StudentID) / Student(StudentID)
5. Enrollment(CourseID) / Course(CourseID)
6. Department(ChairID) / Professor(ProfID)

Part 2

2.1



2.2



Part 4

Task 4.1

1. Functional Dependencies

- 1) StudentID -> StudentName , StudentMajor
 - 2) ProjectID -> ProjectTitle , ProjectType , SupervisorID , StartDate , EndDate
 - 3) SupervisorID -> SupervisorName , SupervisorDept
- Primary Keys are -> StudentID, ProjectID

2. Problems:

- 1) In this table we have repeated information. For example: if several students work on the same project, the project title/type/dates/SupervisionInformation are repeated for every student.
- 2) Update anomaly – If the supervisor's department changes, the department must be updated in every row containing that supervisor. If we missed one row, it creates inconsistent data.
- 3) Insert anomaly – A new project cannot easily be inserted if no student is assigned to it, because StudentID is part of the primary key.
- 4) Delete anomaly – If the only student assigned to a project is deleted, information about the project and its supervisor may also be lost.

3. First Normal Form (1NF)

The table satisfies 1NF if every attribute contains only atomic values and there are no repeating groups. There are no 1NF errors in the given table because each attribute is expected to contain a single value.

4. Second Normal Form(2NF)

We have primary key: StudentID, ProjectID

The primary key has two parts, so we check if some data depends on only one part. Partial Dependencies: StudentID(StudentName, StudentMajor) , ProjectID(ProjectTitle, ProjectType, SupervisorID, StartDate, EndDate). These attributes depend only on part of the composite key. Therefore, the table is not in 2NF.

5. Third Normal Form(3NF)

We have a transitive dependency: ProjectID -> SupervisorID
and: SupervisorID -> SupervisorName/SupervisorDept.

Therefore: ProjectID -> SupervisorName/SupervisorDept.

This is the transitive dependency, so the table is not yet in 3NF. We remove the supervisor information into a separate table.

Final 3NF Decomposition

Student (StudentID [PK], StudentName, StudentMajor)

Supervisor (SupervisorID [PK], SupervisorName, SupervisorDept)

Project (ProjectID [PK], ProjectTitle, ProjectType, SupervisorID [FK], StartDate, EndDate)
StudentProject (StudentID [PK, FK], ProjectID [PK, FK], Role, HoursWorked)

Task 4.2

1. **PRIMARY KEY** of this table: StudentID, CourseID, TimeSlot, Room. Because a single student can be enrolled in multiple courses, so StudentID is not sufficient. And a single course can have different sections, so a CourseID is not sufficient either.

2. **FUNCTIONAL DEPENDENCIES:**

StudentID -> StudentMajor

CourseID -> CourseName

InstructorID -> InstructorName

TimeSlot/Room -> Building

CourseID, TimeSlot, Room -> InstructorID

The last dependence means that the specific course section scheduled for a particular time and classroom has a single instructor.

3. **BCNF – Boyce-Codd Normal Form**

This table is not BCNF, because BCNF requires that the left-hand side of every functional dependency be a superkey.

EXAMPLE: StudentID -> StudentMajor

However StudentKey is not superkey, because single student can have multiple records.

This applies to CourseID/InstructorID too.

4. **BCNF Decomposition**

STUDENT: Student(StudentID[PK], StudentMajor)

COURSE: Course(CourseID[PK], CourseName)

INSTRUCTOR: Instructor(InstructorID[PK], InstructorName)

ROOM: Room(Room[PK], Building) -> Because the rooms across the campus are unique.

SECTION: Section(CourseID[PK], TimeSlot[PK], Room[PK], InstructorID)

ENROLLMENT: Enrollment(StudentID[PK], CourseID[PK], TimeSlot[PK], Room[PK])

5. **Potential Loss of Information**

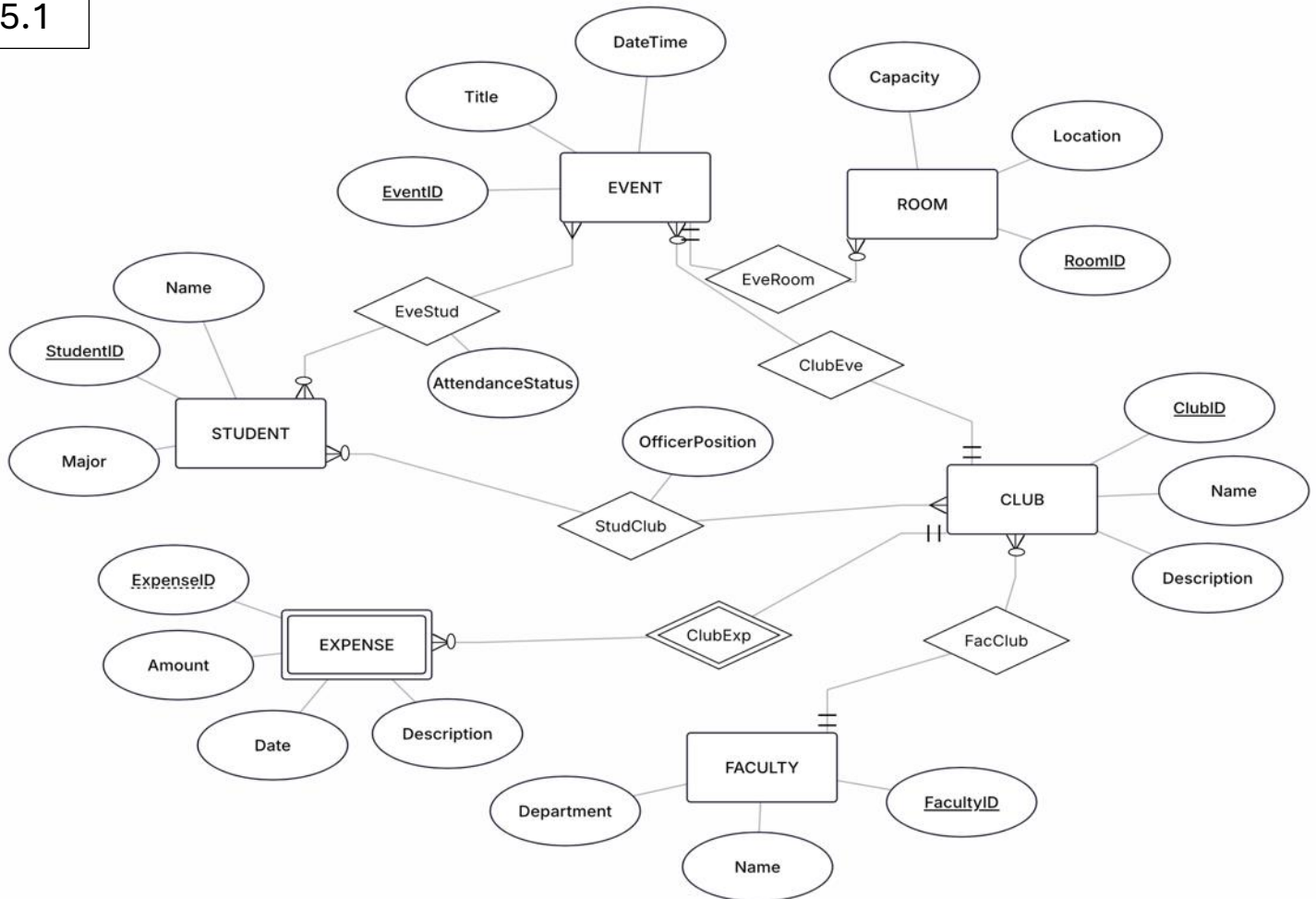
There is no information lost when combining the decomposition results using correct primary and foreign keys.

Reconstruction way: Student + Enrollment + Section + Course + Instructor + Room

For example: The “Enrollment” table shows which student attends which class section, while the “Section” table contains information about the instructor’s name, class time and room number.

Part 5

5.1



STUDENT(StudentID, Name, Major)

CLUB(ClubID, Name, Description, FacultyID[FK] -> FACULTY)

FACULTY(FacultyID, Name, Department)

EVENT(EventID, Title, DateTime, ClubID[FK] -> CLUB, RoomID[FK] -> ROOM)

ROOM(RoomID, Location, Capacity)

EXPENSE(ExpenseID, Amount, Date, Description, ClubID -> CLUB) [PK: ClubID, ExpenseID]

StudClub(StudentID -> STUDENT, ClubID -> CLUB, OfficerPosition) [PK: StudentID, ClubID]

EveStud(StudentID -> STUDENT, EventID -> EVENT, AttendanceStatus) [PK: StudentID, EventID]

3NF TEST

STUDENT: StudentID -> Name, Major. Name and Major doesn't determine anything.

3NF

CLUB: ClubID -> Name, Description, FacultyID. FacultyID is the FK-link, not transitive chain. 3NF

FACULTY: FacultyID -> Name, Department. Name and Department depends only on FacultyID. 3NF

EVENT: EventID -> Title, DateTime, ClubID, RoomID. RoomID and ClubId are just FK-links. And there is no transitive chain. 3NF

ROOM: RoomID -> Location, Capacity. All of them are independent. 3NF

EXPENSE: ClubID/ExpenseID -> Amount, Date, Description. ExpenseID is not unique on its own, so there are no partial dependencies. 3NF

STUDCLUB: StudentID/ClubID -> OfficerPosition. Student holds various position in different clubs, and vice versa. There is only one non-key attribute. 3NF

EVESTUD: StudentID/EventID -> AttendanceStatus. The same logic as a StudClub. 3NF

For club officer roles i had two options

- 1) Add OfficerPosition as an attribute inside the StudClub relationship
- 2) Create a separate OFFICER entity with start/end dates to keep a history of position changes.

I chose first option because it is simpler, and the task only asks to track the current position, not the full history. The downside: if a student's position changes, the old value is just overwritten and lost.

QUERY EXAMPLES

1. Find all students who are officers in the Chess Club
2. List all events for next week, showing the room and the club for each event
3. Show how much a specific club has spent in total